Secure Streams, Adaptive Threads: Enhancing File Transfer Efficiency with Hybrid Compression–Encryption under STEP 2.0

1st Jinyu Cai
School of Advanced Technology
Xi'an Jiaotong - Liverpool University
Suzhou, China
2361409

2nd Jiayi Chu
School of Advanced Technology
Xi'an Jiaotong - Liverpool University
Suzhou, China
2361209

3rd Zhiying Tang
School of Advanced Technology
Xi'an Jiaotong - Liverpool University
Suzhou, China
2360010

4th Jiayu Wang
School of Advanced Technology
Xi'an Jiaotong - Liverpool University
Suzhou, China
2362046

5th Xinyan Xue
School of Advanced Technology
Xi'an Jiaotong - Liverpool University
Suzhou, China
2361538

***Abstract*—This project develops an enhanced STEP2.0 file transfer system based on original STEP protocol with c/s architecture, which integrates LZ78+Huffman compression, AES encryption, and adaptive multi-threading. These functions ensure the system achieves both transmission efficiency and data security. The system achieves automatically choosing thread usage based on file size, supporting folder uploads, and provides real-time progress and upload history tracking. Experimental analysis of various files and folders has proved that the system has achieved reliable, stable and efficient transmission, laying a solid foundation for future improvements.**

***Index Terms*—C/S, Socket, STEP Protocol, TCP, LZ78+Huffman, AES, Multithreading**

I. Introduction

A. Background

FILE transfer protocols play an important role in the management and dissemination of shared information in various distributed systems [1]. From the early development of simple network file exchange systems to the current Secure File Transfer Protocol (SFTP), file transfer systems have continuously evolved from simple host-to-host data movement to secure, efficient, and collaborative platforms that support today's fast-pace global digital infrastructure. According to the CAN201 course learning, our group decided to implement a Simple Transfer and Exchange Protocol (STEP) - an application layer protocol built on TCP/IP - aiming to achieve fast, secure and reliable network file transfer by integrating data compression, encryption and efficient communication mechanisms based on sockets.

B. Task Specification

The existing client-server systems encounter problems such as slow transfer speed of large files and insufficient data security. To solve these problems, our group developed an enhanced STEP2.0 client-server system based on the existing protocol, which integrated data compression and encryption and adaptive muti-threaded mechanisms. The system supports user authentication, multi-threaded transmission, compression detection, AES encryption and decryption, and MD5 integrity verification. It achieves a secure and efficient transmission process through the sequence: compression → encryption → adaptive muti-threaded transmission → decryption → decompression.

C. Practice Relevance

This project can be widely applied to large-scale file transmission, confidential data exchange, providing users with efficient and secure file transfer and management services. By integrating data compression, encryption and adaptive muti-threaded mechanisms, the protocol reduces file size, increases bandwidth usage, and ensuring data confidentiality, integrity, and reliability during transmission.

D. Challenge and Contributions

During the implementation process, we face problems such as compression not always enhancing transmission speed, difficulty in determining the timing of encryption, and adaptive thread numbers. To solve these problems, the client only performs compression when the expected compression ratio exceeds 5%. Verify the encryption correctness by combining AES-256-GCM with MD5 and nonce verification, and adaptively selects 1 to 8 threads based on file size (after compression/encryption) and numbers of file blocks, which achieves a balance between efficiency and security.

II. Related Work

To enhance data transmission efficiency and security, our work priority research in both compression and encryption. The DEFLATE format defines a lossless scheme combining LZ78 sliding-window matching and Huffman coding, achieving high efficiency and platform independence [2]. Huffman's foundational work established the concept of optimal prefix codes minimizing redundancy and average bit length [3]. Further refinements in the ZLIB specification introduced stream-level error detection (Adler-32) and modular support

for DEFLATE blocks [4]. Empirical studies show that hybrid schemes such as LZ78 + Huffman outperform single-method approaches for text and executable files [5]. In encryption, AES has demonstrated superior efficiency and robustness in secure transmission systems [6] [7]. These findings provide a basis for the design of our combined compression and encryption transmission pipeline, which can help the server and client achieve reliable and confidential file transmission.

III. DESIGN

A. Server Debugging

The initial task required resolving critical syntax errors within the provided server-side code to restore its basic functionality and enable subsequent client development.

- Import `struct` package that is required in `make_packet` function for binary data packing operations.
- Change file reading mode in `get_file_md5` function from text mode '`r`' to binary mode '`rb`' to properly handle binary file content.
- Fix variable name case in `data_process` function from lowercase `op_save` to uppercase `OP_SAVE` to match the defined constant.
- Correct logging level from `logger.error` to `logger.info` for normal operation messages in data saving process.
- Fix operation type consistency in file deletion responses from `OP_GET` to `OP_DELETE` to maintain protocol consistency.
- Change socket type in `tcp_listener` function from UDP `SOCK_DGRAM` to TCP `SOCK_STREAM` to align with STEP protocol requirements.
- Fix function call syntax in main function from `tcp-listener` to `tcp_listener` by removing the illegal hyphen.
- Add thread startup by calling `th.start()` method to properly initiate client connection threads.
- Enhance directory creation in the `file_process` function by adding `os.makedirs` calls to ensure proper directory structure for file operations.
- Improve error handling in file upload process with better exception management and status message reporting.

B. C/S network Architecture Design

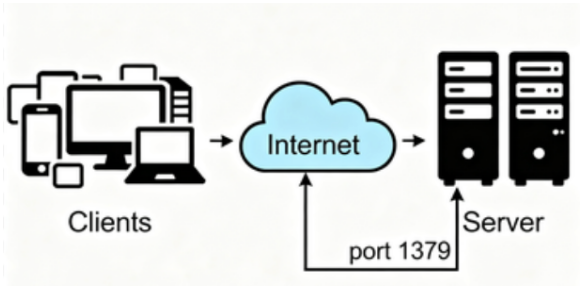

Fig. 1. Client and Server Architecture

This project implements a client-server architecture based on the STEP protocol for reliable file transfer. As shown in Fig. 1, the server listens on a specific TCP port and handles multiple concurrent client connections through multi-threading. The system employs a structured binary protocol that encapsulates JSON metadata and file data into unified packets, ensuring secure and efficient communication through token-based authentication and block-level file transfer mechanisms. The architecture supports several advanced features, including intelligent compression, AES encryption for data security, and adaptive thread allocation based on file characteristics. The server automatically processes encrypted and compressed files during upload, providing a robust foundation for secure file transfer operations.

C. Client Sequence Diagram

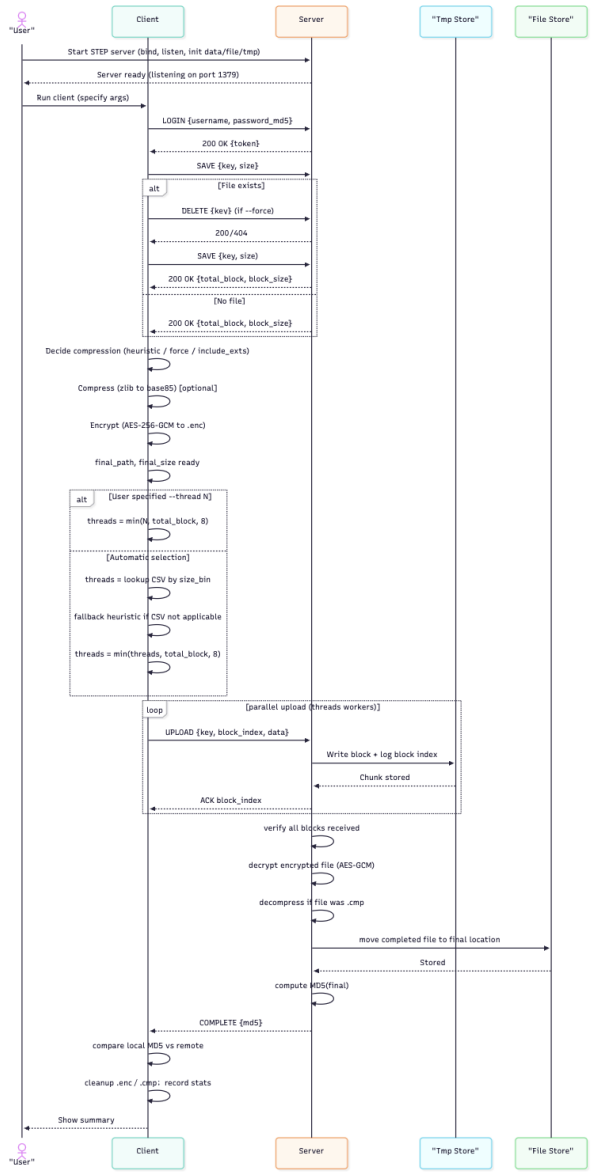

Fig. 2. Client-Server Sequence Diagram

D. Key Functional Design

1) *File Transfer Design*:

- **Adaptive Multithreaded Upload Mechanism:** We use single-threaded upload for small files or a small number of data blocks. Moreover, large files will automatically enable a multi-threading mechanism that adaptively selects the number of threads based on file size and the number of blocks. We created a CSV file for recording best results. However, since the file cannot be submitted, the system will apply a predefined rule to determine the thread number according to the file size.
- **Compression and Encryption Integration:** The file is compressed by LZ78 + Huffman and encrypted with AES before transmission to ensure data security. On the server side, the AES decryption and decompression processes will be carried out. This can ensure data confidentiality.
- **Folder upload:** Based on the single-file upload function, we achieve user to upload folder. When they input the `--folder` argument, all files under the target folder are sequentially processed through the same pipeline of compression and AES encryption before being upload.

2) *Transfer Feedback and Debugging*:

- **Real-time Transfer Progress Display:** The client provides real-time transfer progress, showing chunk completion percentage, current thread status, and total running time to user.
- **Historical Record Tracking:** After each successful transmission, the system will record the uploaded file name, size, transmission time, verification and verification result, which helps to facilitate subsequent verification and performance comparison.

E. Algorithm

The content of the following code blocks are the kernel pseudo-code for user authorization and file uploading.

Algorithm 1: User Authentication

```
Input: username, server_ip, server_port
Output: authentication_token

password ← MD5_Hash(username)
login_json ← {username, password, operation:
  LOGIN, type: AUTH}
sock ← TCP_Connect(server_ip, server_port)
message ← Make_STEP_Packet(login_json)
sock.Send(message)
response ← sock.Receive()
json_data ← Parse_STEP_Packet(response)

if json_data.status == 200 then
    token ← json_data.token
    Print "Token: {token}"
    return token
else
    Print "Authentication Failed"
    return None
```

Algorithm 2: File Upload

```
Input: file_path, key_name, token
Output: upload_success, performance_metrics

// Phase 1: File Preprocessing
file_size ← Get_File_Size(file_path)
compressed_path, compressed ←
  Smart_Compress(file_path)
encrypted_path, encrypted ←
  AES_Encrypt(compressed_path)

// Phase 2: Server Coordination
sock ← TCP_Connect(server_ip, server_port)
upload_plan ← Request_Upload_Plan(file_size,
  key_name, token)
if upload_plan.status == 402 then
    if User_Confirms_Overwrite() then
        Delete_Remote_File(key_name, token)
        upload_plan ←
        Request_Upload_Plan(file_size, key_name,
        token)

// Phase 3: Multi-threaded Block
   Upload
total_blocks ← upload_plan.total_block
thread_count ← Adaptive_Thread_Count(file_size,
  total_blocks)
UPLOAD_BLOCK ← 0
LOCK ← Thread_Lock()

for i ← 0 to thread_count−1 parallel do
    thread_sock ← TCP_Connect(server_ip,
     server_port) Start_Thread(Upload_Worker,
     thread_sock, file_path, key_name, total_blocks,
     UPLOAD_BLOCK, LOCK)

Wait_All_Threads_Complete()

// Phase 4: Integrity Verification
remote_md5 ← Get_Last_Block_Response().md5
local_md5 ← Calculate_File_MD5(file_path)
if local_md5 == remote_md5 then
    Cleanup_Temporary_Files()
    return True, Calculate_Metrics()
else
    Trigger_Retransmission()
    return False, None
```

IV. IMPLEMENTATION

A. Develop Environment

This project was implemented in a Python-based environment.

TABLE I
WINDOWS DEV ENVIRONMENT

OS	Microsoft Windows 11
CPU	i5 (12 cores)
RAM	16GB@4800 MHz
IDE	PyCharm 2023.2.1
Python	Python 3.11.5

Key libraries used include:

- *socket*:enables stable TCP communication between server and client.
- *threading*:supports concurrent uploads of file segments.
- *hashlib*:achieves MD5 checksum validation to ensure data integrity.
- *os*:handles file management and path operations.
- *json*:structures data encoding.
- *pycryptodome*:implements AES encryption and decryption for secure transmission.

B. Programming Skills

1) **Object-Oriented Programming (OOP)**: Classes encapsulate core functionalities of both client and server, organizing operations such as network connection, protocol parsing, and file processing into object methods. This approach enhances code reusability and extensibility through inheritance and polymorphism.
2) **Multi-threading**: Implemented multi-threading to enhance the efficiency of file transfers by uploading file blocks concurrently. This reduced wait times and improved overall system performance while maintaining thread safety.
3) **Global Lock Design**: Implemented a mutex-based global lock mechanism to avoid read-and-write errors during concurrent operations and ensure safe access.
4) **Error Handling**: Established comprehensive error handling mechanisms throughout the system. Implemented exception handling, resource cleanup, and user-friendly error reporting to ensure system stability and reliability.
5) **Low-Coupling Architecture**: Clear interface specifications and abstraction layers establish loose coupling between system components, reducing system complexity and enhancing maintainability.

C. Steps of Implementation

1) **STEP Protocol and Requirements Analysis:** Analyze and clarify the core functions that the system needs to implement based on the STEP protocol provided in the LMO.
2) **Restore and Debug server:** Debug the original errors based on the given server-side code, then complete the log system and multi-threaded connection processing, providing a stable server-side environment for later process.
3) **Implement the login authentication:** The client establishes a TCP connection, generates a login request, and extracts the Token from the server response.
4) **File compression processing:** By combining zlib compression with base85 encoding to simulate the LZ78 and Huffman algorithms to compress the needed files.
5) **File encryption:** Using the AES algorithm to achieve file encryption and decryption. The client encrypts and uploads, and the server automatically recognizes and decrypts it.
6) **Multithreaded upload:** Processing file block uploads concurrently through multiple threads, and an adaptive thread selection mechanism is used to automatically optimize the number of threads.
7) **Collect and organize the uploaded data:** Record the throughput rate, latency, packet loss rate and other indicators of the uploaded files process, and save the results to a CSV file for future analysis.
8) **Improve exceptions and logs:** Handle possible exceptions that may occur during the addition process and record the running status and error information through the log system.

D. Program Flow Chart

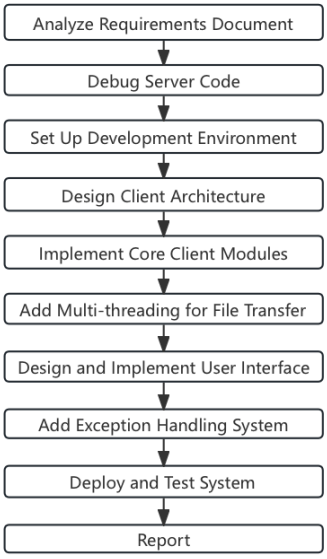

Fig. 3. Flowchart of Implementation.

E. Program Implementation

- **Authorization Function:** This function enables the client to log in to the STEP server and obtain the Token required for authorization operations. When user logs in with id, the client will establish a TCP connection through the `get_sock()` function and generate the login request packet in the `login()` function. Subsequently, the login package is encapsulated through the `make_message()` function and sent to the server. After the server verifies, it will return a response package containing the authorization Token. The client receives and parses the response, using `parse_message_from_server()` function to extract the Token and saving it in the global variable for identity verification in later operations.

- **File Uploading Function:** This function enables server and client to work together to complete transmission and verification. Firstly, client uses `save_file()` function to send file transmission plan to server in JSON format, which contains the file size and file name as the unique identifier "key". After the server receiving, it will return plan and inform the client that the file will be divided into the numbers of blocks and the size of each block. Then, the client checks the response status and read and split the file into blocks after confirming the file does not exist on the server yet. Each block is encapsulated as upload request using `make_message()` function, and then transmitted to the server in sequence through single-threaded `upload_single()` or multi-threaded `upload_multi()`. After all the data blocks have been transmitted, the server will merge the files and perform decryption and decompression, finally returns the MD5 check with the files. The client will compare the MD5 returned by the server. If they are consistent, which indicates that the upload was successful, and outputs the transmission rate, delay and success status information in the terminal.

F. Difficulties and Solutions

1) Compressing pre-compressed files (e.g., JPG, PDF, DOCX) using DEFLATE may increase their size, as these formats are already optimized [5]. Since no lossless algorithm guarantees reduction in all cases, the system adaptively skips compression when size increases. File extensions and size comparisons guide this decision. For testing, options like – force_compress and – include_exts allow overriding this behavior.
2) To enhance file management efficiency, the system was designed to handle duplicate files during upload by providing users with options to either overwrite or skip existing files. Additionally, folder upload capability was implemented to support batch file processing, eliminating the previous limitation of single-file-only transfers.

V. TESTING AND RESULTS

A. Testing Environment & Metrics

TABLE II
TESTING ENVIRONMENT

OS	Microsoft Windows 10 pros
CPU	i9-12950HX
RAM	64GB@4800 MHz
IDE	PyCharm 2024.2
Python	Python 3.11.7

All tests were conducted on a local host server using the STEP2.0 system. The evaluation metrics include total upload time, throughput (MB/s), average delay per block (ms), packet loss rate (%), and MD5 verification result. These metrics are automatically logged in `data/<user_id>/records.csv` after each upload.

B. Testing Steps

The testing process was divided into several phases to verify the functionality, performance, and reliability of the STEP system.

1) Server Setup: After entering "python server.py" in the terminal. The server is ready to be connected, andthe message "Server is ready" is displayed in the terminal, as shown in Fig. 4.

Fig. 4. Server is ready

2) Client Authentication: The client connected server through `python client.py --server_ip --id` . Successful login returned a token verifying authentication before data transfer began.

3) Compression and Encryption Test: The compression and encryption was tested using both compressible and already-compressed files. As shown in Fig. 5, the client system dynamically determines whether compression should be applied. For normal binary data like `test.bin` the system successfully compressed the file from 10.24 MB to 12.46 KB before AES-GCM encryption, achieving a compression ratio of 99.88%. For already-compressed files, the system automatically skipped the compression phase and only performed encryption, with a minimal overhead of +0.01%. This adaptive compression ensures optimal efficiency without redundant processing.

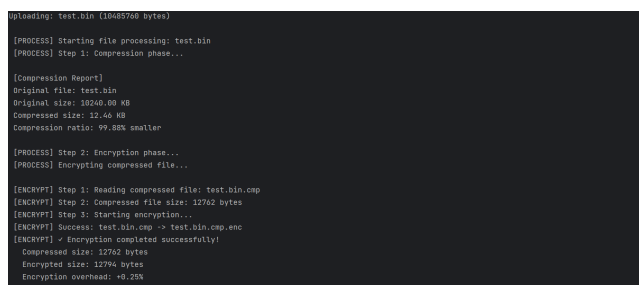

Fig. 5. Compression and encryption test results for `test.bin`

4) Multi-threading Verification: To evaluate adaptive thread scheduling, files of various sizes (from 0.1 MB to above 128 MB) were uploaded while varying the thread count from 1 to 8. As shown in Fig. 6, the client dynamically selected the number of threads using the `resolve_thread_count()` function based on file size and historical performance records. For each configuration, processing time and throughput were recorded automatically.

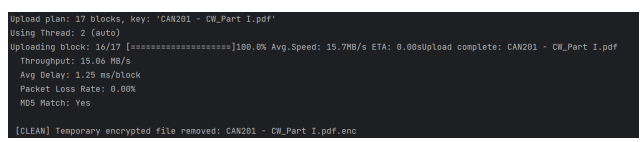

Fig. 6. Adaptive thread upload test

5) History and Log Verification: After multiple uploads, the command `--history` was executed to print the last 7 upload records. The output table is shown in Fig. 7, which contains transmission details and status of each file.

```
PS D:\CAN201\CAN201> python client.py --server_ip 127.0.0.1 --id 2360010 --history

Recent 7 upload records for user 2360010:

+------------------------+----------+------+------------+-------+------+-----+---------+
| File                   | Size     | Time | Throughput | Delay | Loss | MD5 | Status  |
+------------------------+----------+------+------------+-------+------+-----+---------+
| CAN201 - CW_Part I.pdf | 334918   | 0.03 | 9.8        | 1.92  | 0.0  | Yes | Success |
| client.py              | 45747    | 0.01 | 1.86       | 7.98  | 0.0  | Yes | Success |
| test.bin               | 10485760 | 0.04 | 0.27       | 44.49 | 0.0  | Yes | Success |
| xjtlu.jpg              | 9451     | 0.01 | 1.5        | 6.03  | 0.0  | Yes | Success |
| sub_folder/test.bin    | 10485760 | 0.04 | 0.3        | 40.33 | 0.0  | Yes | Success |
| test.bin               | 10485760 | 0.04 | 0.31       | 39.21 | 0.0  | Yes | Success |
| CAN201 - CW_Part I.pdf | 334918   | 0.03 | 9.86       | 1.91  | 0.0  | Yes | Success |
+------------------------+----------+------+------------+-------+------+-----+---------+
```

Fig. 7. Upload history record

C. Testing result

Overall, the results demonstrate that:

1) Compression Efficiency: For raw files, compression reduced its size by more than 99%, while pre-compressed formats files were automatically skipped to save time. The AES encryption overhead remained below 0.05%, which demonstrates no such impact on total transfer time.

2) Multi-threading Performance Evaluation: Files ranging from 0.1 MB to above 128 MB were uploaded with thread numbers from 1 to 8. We analyze the adaptive thread mechanism by calculating the average processing time and throughput within each file size range. The optimal threads selected for different size files are shown in Figs. 8, 9, and 10.

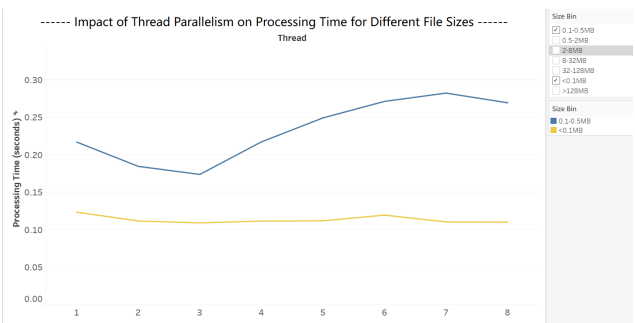

Fig. 8. Thread parallelism impact on small files (<0.5 MB)

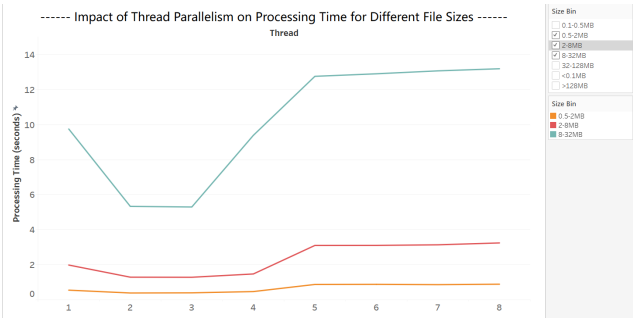

Fig. 9. Thread parallelism impact on medium files (0.5–32 MB)

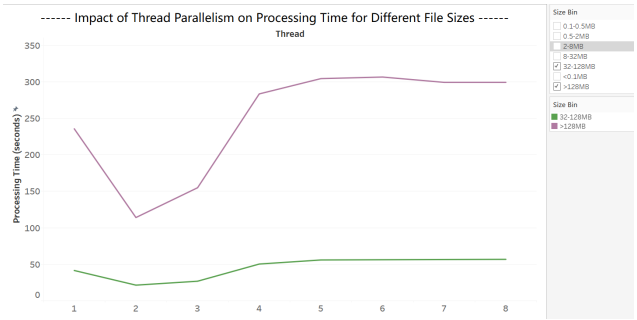

Fig. 10. Thread parallelism impact on large files (>32 MB)

Based on the experimental results, we find that using 2–3 threads provides the optimal upload performance regardless of file size. This is because adding a new thread requires the operating system to create a block and allocate stack space. When the numbers of threads exceed 3, the overhead of context switching more over than the performance on concurrency, leading to bad performance on multi-threading.

VI. CONCLUSION

This project successfully developed and evaluated an enhanced file transfer system based on the STEP protocol, integrating advanced features such as compression, encryption, adaptive multi-threading, and supporting folder upload. The implementation ensures both transfer efficiency and data security under variable file sizes. Looking ahead to future development, we hope to achieve a fully adaptive thread management module that can achieve real-time dynamic adjustment based on file size, bandwidth and latency feedback. In conclusion, this work established a solid foundation for a secure and efficient file transfer platform.

VII. ACKNOWLEDGMENT

During the project collaboration, all members contributed equally by actively participating in discussions and completing tasks responsibly. Therefore, we totally agree that each our team member made an equal contribution to this project.

REFERENCES

[1] P. F. Linington, "File transfer protocols," *IEEE J. Sel. Areas Commun.*, vol. 7, no. 7, pp. 1052–1059, Sep. 1989.

[2] L. P. Deutsch, *DEFLATE Compressed Data Format Specification Version 1.3*, RFC 1951, Aladdin Enterprises, May 1996.

[3] D. A. Huffman, "A Method for the Construction of Minimum-Redundancy Codes," *Proc. IRE*, vol. 40, no. 9, pp. 1098–1101, Sep. 1952, doi: 10.1109/JRPROC.1952.273898.

[4] L. P. Deutsch and J.-L. Gailly, *ZLIB Compressed Data Format Specification Version 3.3*, RFC 1950, Aladdin Enterprises and Info-ZIP, May 1996.

[5] A. V. Vashchenko, I. E. A. Drozdova, and O. O. Prykhodko, "Determining Optimal Compression Algorithm for Files of Different Formats," *Visnyk Kherson Nat. Tech. Univ.*, no. 2, pp. 126–129, 2024, doi: 10.35546/kntu2078-4481.2024.2.17.

[6] J. Dutt, A. Kumar, and N. Pratap, "Comparative Study on Different Encryption and Decryption Algorithm," in *Proc. Int. Conf. Adv. Comput. Innov. Technol. Eng. (ICACITE)*, 2021, pp. 20–22, doi: 10.1109/ICACITE51222.2021.9404734.

[7] O. M. E. Mahdi and J. Juremi, "EFTS: An Encryption File Transfer System Applying Advanced Encryption Standard (AES) Algorithm," *Int. J. Data Sci. Adv. Anal.*, vol. 4, Special Issue 1, pp. 221–223, 2022.